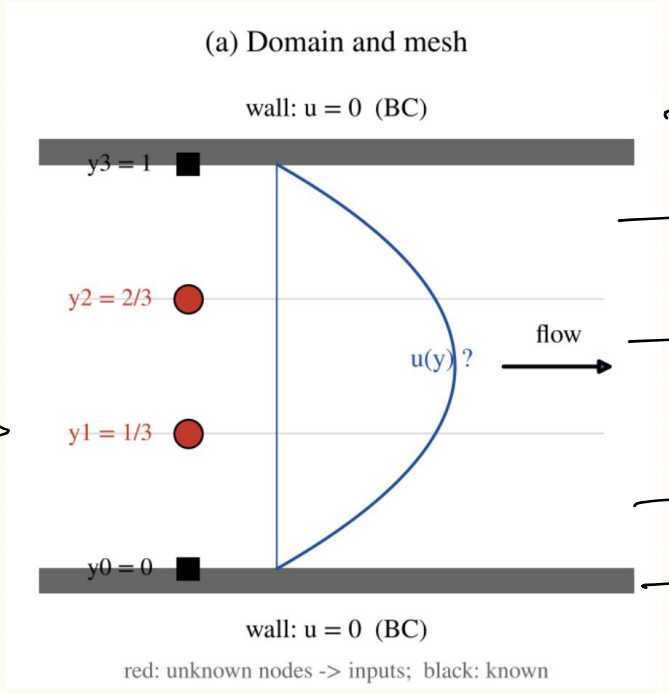
Flow Between 2 flat plates



wall velocities → $u_0 = 0$ Boundary con

3 cells

2 unknowns

BC $u_0 = 0$

$0 \le y \le 1$

fully developed laminar flow

Domain and physics

#1

x-momentum:

$$\rho\left(\partial u/\partial t + u\,\frac{\partial u}{\partial x} + v\,\partial u/\partial y\right) = -\partial p/\partial x + \mu\left(\frac{\partial^2 u}{\partial x^2} + \partial^2 u/\partial y^2\right)$$

Fully developed laminar flow:

1. steady $\partial u/\partial t = 0$

2. The profile no longer changes along the channel

$\partial u/\partial x = 0$ $\quad \partial^2 u/\partial x^2 = 0$

3. Continuity $\partial u/\partial x + \partial v/\partial y = 0$ gives $\partial v/\partial y = 0$

since $v = 0$ at walls

$v = 0$ everywhere

4. Inertia and convection 0. u depends on y only.

Remains the force Balance:

$$0 = -dp/dx + \mu \, d^2u/dy^2$$

$$\Rightarrow \frac{d^2u}{dy^2} = \frac{1}{\mu}\left(dp/dx\right)$$

$$= G_1$$

dp/dx = constant — only depends on x

G_1 is negative because pressure falls in the flow direction.

#2 Dimensionless form

Measure y in units of the gap height H

So, $0 \le y \le 1$

$$U_{ref} = -G_1 H^2/2 \quad \text{---(1)}$$

2 is chosen to get $y(1-y)$ if $-G_1H^2$ chosen from $y(1-y/2)$

Integrate twice

$$du/dy = -2y + C_1$$

$$u = -y^2 + C_1 y + C_2$$

Applying Boundary Conditions (no slip)

At, $y = 0$, $u = 0$

$C_2 = 0$

At, $y = 1$

$u = 0$

$0 = -1^2 + C_1 \cdot 1 + 0$

$C_1 = 1$

$$u(y) = -y^2 + y$$
$$= y(1-y)$$

Exact answer

$u(1/2) = 1/2(1-1/2) = 1/4$ (at the center)

mean velocity $\int_0^1 y(1-y)\,dy = 1/6$

Poiseuille flow: pressure driven laminar flow named after Jean Poiseuille

The ratio $= {}^{1/4}/_{1/6} = 1.5$ is known as Poiseuille ratio

$$= \frac{\text{fastest speed at the center}}{\text{Average Speed}}$$

from (1)

$U_{ref} = -GH^2/\nu$

$y = H \cdot \hat{y}$ y runs from 0 to 1

$0 \le y \le 1$

$\hat{y} = y/H$ Taking a gap $H = 0.01$ m (cell height) or 1cm

at bottom wall (0m)	$\hat{y} = 0$ (min)
Middle (0.005m)	$\hat{y} = 0.5$
Top wall (0.01m)	$\hat{y} = 1$ (max)

Reference Speed U_{ref} if $U_{ref} = 0.05$ m/s

$\hat{u} = u/U_{ref}$ a real speed of 0.0125 m/s is $0.0125/0.05 = 0.25$ m/s

Second derivative gives U_{ref}/H^2

derivation of (1)

change in u per change in y

Changing the y-unit: 1 step in $\hat{y}$ is a step of H meters.

Per meter $= 1/H \times$ per unit "y"

first derivative gives $1/H$

second derivative gives $1/H^2$ (slope of the slope)

$$\frac{d^2u}{dy^2} = \left(\frac{U_{ref}}{H^2}\right) \cdot \frac{d^2\hat{u}}{d\hat{y}^2} \quad \left| \frac{d^2u}{dy^2} = G \right.$$

$$\frac{d^2\hat{u}}{d\hat{y}^2} = G \cdot H^2 / U_{ref}$$

check from exact answer

$$u = U_{ref} (y/H)(1 - y/H) = U_{ref}(y/H - y^2/H^2)$$

$$du/dy = U_{ref}(1/H - 2y/H^2)$$

$$d^2u/dy^2 = U_{ref}(-2/H^2) = -2U_{ref}/H^2$$

$$\frac{d^2u}{dy^2} = G \qquad -2U_{ref}/H^2 = G$$

$$U_{ref} = -GH^2/2 \quad \text{——(1)}$$

PDE: $\frac{d^2u}{dy^2} = -2$

BC $u(0) = 0$, $u(1) = 0$ (no slip)

Exact solution: $u = y(1-y)$

Step 2 # Mesh : The Domain decides the inputs

Table 1

3 cells, h = 1/3, nodes $y_0 = 0$, $y_1 = 1/3$, $y_2 = 2/3$, $y_3 = 1$.

node	y	u	role
0	0	0	wall: known from the BC, not an input
1	1/3	?	unknown → **input 1: y = 0.3333**
2	2/3	?	unknown → **input 2: y = 0.6667**
3	1	0	wall: known from the BC, not an input

So the **input vector is just the positions of the unknown nodes**: x = (1/3, 2/3).

Step 3 # The PDE on the mesh

Central difference at each unknown nodes

$u_0 = u_3 = 0$

$1/h^2 = 9$

node j

$$\frac{u_{j-1} - 2u_j + u_{j+1}}{h^2} = -2$$

node 1 at $y = 1/3$

neighbours node 0 and 2

$$\frac{u_0 - 2u_1 + u_2}{h^2} + 2 = 0$$

$$r_1 = 9(-2u_1 + u_2) + 2$$

r_2 at node 2 $y = 2/3$

$$\frac{u_1 - 2u_2 + u_3}{h^2} + 2 = 0$$

$$r_2 = 9(u_1 - 2u_2) + 2$$

r is the Residual it measures how far a guess for u_1 and u_2 is satisfying

$r = 0$ (The guess is the solution)

$r \neq 0$ (guess is wrong)

Some random guess

guess u_1	guess u_2	$r_1 = 9(-2u_1 + u_2) + 2$	$r_2 = 9(u_1 - 2u_2) + 2$	meaning
0	0	$9(0) + 2 = \mathbf{2}$	$9(0) + 2 = \mathbf{2}$	no flow, too slow
0.1	0.1	$9(-0.2 + 0.1) + 2 = -0.9 + 2 = \mathbf{1.1}$	$9(0.1 - 0.2) + 2 = \mathbf{1.1}$	still too slow
0.3	0.3	$9(-0.6 + 0.3) + 2 = -2.7 + 2 = \mathbf{-0.7}$	$\mathbf{-0.7}$	too fast, overshoot
2/9	**2/9**	$9(-4/9 + 2/9) + 2 = -2 + 2 = \mathbf{0}$	$9(2/9 - 4/9) + 2 = \mathbf{0}$	**correct**

+ r means fluid is too slow to push

$u_1 = u_2 = 2/9$ make the both residuals 0

Step 4

forward pass: input → hidden → output

1. Input: the position y
2. Hidden layer: 2 nodes.

from Table 1

$s_1 = 3 \cdot y - 1$

$w_1^* = 3$ $b_1 = -1$

node 1 $y = 1/3$ $a_1 = \tanh(s_1)$

Row 1: input y = 1/3 (node 1 of the mesh)

step	calculation	result
hidden node 1	$s_1 = 3 \cdot (1/3) - 1 = 1 - 1$	**0**
	$a_1 = \tanh(0)$	**0**
hidden node 2	$s_2 = 3 \cdot (1/3) - 2 = 1 - 2$	**−1**
	$a_2 = \tanh(-1)$	**−0.76159**
output	$u_1 = z_1 \cdot 0 + z_2 \cdot (-0.76159)$	$\mathbf{u_1 = -0.76159 \cdot z_2}$

Row 2: input y = 2/3 (node 2 of the mesh)

step	calculation	result
hidden node 1	$s_1 = 3 \cdot (2/3) - 1 = 2 - 1$	**1**
	$a_1 = \tanh(1)$	**0.76159**
hidden node 2	$s_2 = 3 \cdot (2/3) - 2 = 2 - 2$	**0**
	$a_2 = \tanh(0)$	**0**
output	$u_2 = z_1 \cdot 0.76159 + z_2 \cdot 0$	$\mathbf{u_2 = 0.76159 \cdot z_1}$

Output: $u = z_1 \cdot a_1 + z_2 \cdot a_2$

$s_2 = 3 \cdot y - 2$

weight $w_2 = 3$

bias $b_2 = -2$

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

$x = 1$

$$\tanh(1) = \frac{2.71828 - 0.36788}{2.71828 + 0.36788}$$

$$= 0.76159$$

The weights and biases are random.

Z = output weights

What each value of z gives

z_1	z_2	u_1 (at y = 1/3)	u_2 (at y = 2/3)	flow?
0	0	0	0	no flow (starting guess)
0.1	−0.1	0.076	0.076	too slow
0.29179	**−0.29179**	**0.2222**	**0.2222**	**correct**
0.5	−0.5	0.381	0.381	too fast

→ This only setting makes the residual 0

Z found from the physics. Z is chosen so that the PDE on the mesh is satisfied ($r_1 = r_2 = 0$)

In normal Neural Network output weight (Z) is learned by backpropagation on data.

frozen: Hidden weights w, b are frozen
In code it is controlled by seed.

$$u(y) = z_1 \tanh(w_1 y + b_1) + z_2 \tanh(w_2 y + b_2)$$

At the beginning w and b are drawn by program from random number generator and only once. They are frozen and fixed by the seed value.

freezing the w and b makes the hidden-layer values a_1 and a_2 at every input point fixed numbers. Then $u = z_1 a_1 + z_2 a_2$ is a plain linear sum in z and solving z becomes ordinary linear algebra

How z converges steps:

1. Start: $z = (0, 0)$ so, $u = (0, 0)$ $r = (2, 2)$ (wrong)

2. How r reacts on z.
Since a_1 and a_2 are frozen we can write r directly in terms of z.

$$r_1 = 6.854 z_1 + 13.709 \cdot z_2 + 2$$
$$r_2 = -13.709 \cdot z_1 - 6.854 \cdot z_2 + 2$$

The numbers come from sensitivity table which is Jacobian $J = \partial r / \partial z$

3. **Newton Steps:** find the change Δz that makes r zero.

Solve $J \cdot \Delta z = -r$:

$$6.854 \cdot \Delta z_1 + 13.709 \cdot \Delta z_2 = -2$$

$$-13.709 \cdot \Delta z_1 - 6.854 \cdot \Delta z_2 = -2$$

This gives $\Delta z_1 = 0.29179$ $\Delta z_2 = -0.29179$

Update: $z = 0 + \Delta z = (0.29179, -0.29179)$

So, $u(0.2222, 0.2222)$ and $r = (0, 0)$

Solution converged.

why random: w, b still gives correct ans:-

change $a_1 = \tanh(6 \cdot y - 3)$ $a_2 = \tanh(-3y + 1)$

$z = (-0.29179, -0.58357)$

but, u still same $u_1 = u_2 = 0.2222$

The random choices change z but not the answer. Answer is decided by PDE and BC

The Jacobian $J = \partial r / \partial z$

$a = \tanh 1 = 0.76159$

$r_1 = 9(-2u_1 + u_2) + 2$

$r_2 = 9(u_1 - 2u_2) + 2$

$u_1 = -a \cdot z_2 \qquad u_2 = a \cdot z_1$ (Table 1)

from the forward pass

Substitute into r:

$r_1 = 9(-2 \cdot (-az_2) + az_1) + 2 = 9 \cdot a \cdot z_1 + 18 \cdot a \cdot z_2 + 2$

$r_2 = 9(-az_2 - 2 \cdot a \cdot z_1) + 2 = -18a \cdot z_1 - 9 \cdot a \cdot z_2 + 2$

$\frac{\partial r_1}{\partial z_1}$	$9 \cdot a$	$9 \times 0.76159 = 6.859$
$\frac{\partial r_1}{\partial z_2}$	$18 \cdot a$	13.709
$\frac{\partial r_2}{\partial z_1}$	$-18 \cdot a$	-13.709
$\frac{\partial r_2}{\partial z_2}$	$-9 \cdot a$	-6.859

The Jacobian

residual

$$J = \partial r / \partial z = \begin{vmatrix} 6.854 & 13.709 \\ -13.709 & -6.854 \end{vmatrix} \begin{matrix} \rightarrow r_1 \\ \rightarrow r_2 \end{matrix}$$

$\downarrow z_1$ $\downarrow z_2$

Size = (number of equations) × (number of unknowns)

2 × 2 (here)

The Newton step is a Matrix operation:

$J \cdot \Delta z = -r$:

$$J \cdot \begin{vmatrix} \Delta z_1 \\ \Delta z_2 \end{vmatrix} = \begin{vmatrix} -2 \\ -2 \end{vmatrix}$$

$\Delta z = (0.29179, -0.29179)$

$J = S \cdot \phi$

$S = \partial r / \partial u = \begin{vmatrix} -18 & 9 \\ 9 & -18 \end{vmatrix}$

hidden layer output ϕ

$$\phi = \partial u / \partial z = \begin{vmatrix} 0 & -0.76159 \\ 0.76159 & 0 \end{vmatrix}$$

ϕ is the network matrix

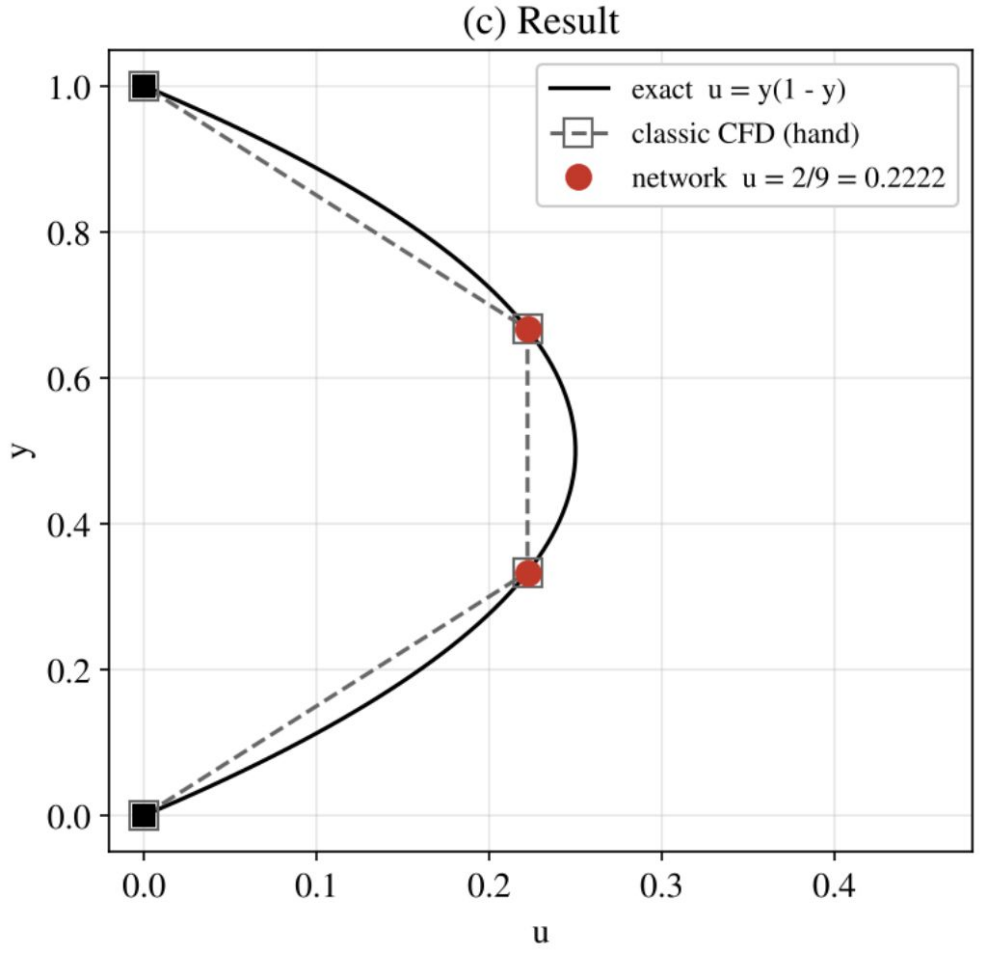
$u(y) = z_1 \cdot \tanh(3y-1) + z_2 \cdot \tanh(3y-2)$

$\partial u / \partial z_1 = \tanh(3y-1)$ $\partial u / \partial z_2 = \tanh(3y-2)$

Putting into $\tanh(s) = \frac{e^s - e^{-s}}{e^s + e^{-s}}$

The final Result Comparison :

node	network: $u = z_1 a_1 + z_2 a_2$	classic CFD (hand)	exact y(1−y)	residual r
y = 1/3	0.29179·0 + (−0.29179)(−0.76159) = **0.2222**	0.2222	0.2222	0
y = 2/3	0.29179·0.76159 + (−0.29179)·0 = **0.2222**	0.2222	0.2222	0



SPIRAL: physics-solved output weights on a frozen random tanh layer

No training data. The network supplies the unknown field; the discretised PDE with its boundary conditions decides the output weights by Newton's method.

1 Domain & mesh — Ω, grid, BCs. Unknowns sit at positions x_i; boundary values are known.

→

2 Input — $X = [x_1; \ldots; x_n]$, one row per unknown (its coordinates).

→

3 Hidden layer (frozen) — $\Phi_{ik} = \tanh(w_k \cdot x_i + b_k)$; w, b random, never updated.

→

4 Output — $u = \Phi z$ (in 3-D: $A = \Phi z$, $u = \text{curl } A$). Only z is unknown.

→

5 Physics residual — r(u): PDE written with finite differences at every unknown, BC values inserted.

→

6 Newton on z — $J = S\Phi$, solve $J\Delta z = -r$ (kernel form), $z \leftarrow z + \Delta z$.

↶ repeat steps 4–6 until $r \approx 0$ (linear PDE: one step; Navier–Stokes: 2–7 steps per time step)

Algorithm 1 SPIRAL solver (steady or one time step; the time loop repeats lines 7–15)

Input: domain Ω, mesh, PDE, boundary conditions, number of hidden nodes P, steepness range $[s_{min}, s_{max}]$, seed, tolerance ε, damping λ

Output: output weights z; field $u(x) = \Sigma_k z_k \tanh(w_k \cdot x + b_k)$ (3-D: $A(x)$, $u = \text{curl } A$)

1 *// set-up: from the domain to the network input*
2 Mesh Ω; list the n unknowns and their positions $x_1, \ldots, x_n$; stack them as rows of $X \in \mathbb{R}^{n \times d}$ *// boundary values are not inputs*
3 **for** k = 1,…,P **do** *// random hidden layer, drawn once with the seed*
4 $s_k \sim$ log-uniform$[s_{min}, s_{max}]$; $d_k \sim$ random unit vector; $x0_k \sim$ uniform in Ω
5 $w_k \leftarrow s_k d_k$; $b_k \leftarrow -w_k \cdot x0_k$ *// a smooth step through $x0_k$, steepness s_k*
6 $\Phi \leftarrow \tanh(XW + b) \in \mathbb{R}^{n \times P}$; **freeze** W, b, Φ *// never changed again*
7 Build the discrete residual r(u): PDE by finite differences at each unknown, BC values inserted *// the "loss"*
8 $z \leftarrow 0$ (or the previous time level); $u \leftarrow \Phi z$; $r \leftarrow r(u)$
9 **repeat** *// Newton iterations on the output weights only*
10 $S \leftarrow \partial r/\partial u$ *// physics Jacobian: the CFD matrix (sparse)*
11 $J \leftarrow S\Phi$ *// chain rule: $\partial r/\partial z = (\partial r/\partial u)(\partial u/\partial z)$*
12 Solve $(JJ^T + \lambda I)\,\alpha = -r$ by Cholesky; $\Delta z \leftarrow J^T\alpha$ *// smallest Δz with $J\Delta z = -r$*
13 $z \leftarrow z + \Delta z$; $u \leftarrow \Phi z$; $r \leftarrow r(u)$
14 **until** $\|\Phi\Delta z\|_\infty / \|\Phi z\|_\infty < \varepsilon$ *// converged: the discrete PDE holds*
15 **return** z, $u = \Phi z$ (3-D: $u = C\Phi z$, C = discrete curl)

Term	Meaning	1-D hand example	SPIRAL-3D code (24×24×2)
Ω, mesh	The flow region and its grid.	gap $0 \le y \le 1$, 3 cells, h = 1/3	cavity, 24 × 24 × 2 cells
BC	Known values on the boundary. They are written into r (wall or ghost values), never fed to the network.	u(0) = u(1) = 0	no-slip walls; lid speed U enters the lid rows of r
x_i, X	**Input**: the position of the i-th unknown; X stacks all of them.	y = 1/3, 2/3 (n = 2)	(x, y, z) of 2,162 grid edges
w_k, b_k	**Hidden weights and biases.** Random, drawn once from a seed, then **frozen**: no backprop, no update, ever.	w = (3, 3), b = (−1, −2)	1,441 nodes, |w| in [2, 84]
Φ	**Hidden-layer output matrix**, $\Phi_{ik} = \tanh(w_k \cdot x_i + b_k)$. Fixed because w, b are frozen. Equal to $\partial u/\partial z$.	[[0, −0.762], [0.762, 0]]	one block per output: (552, 552, 1,058) × 1,441
z	**Output weights: the only unknowns.** Start at 0, solved from the physics.	(0.29179, −0.29179)	3 × 1,441 = 4,323 numbers
u = Φz	Network output: the field at the unknown points (3-D: vector potential A, velocity u = curl A, so div u = 0 exactly).	$u_1 = u_2 = 0.2222$	A on edges → u on faces
r	**Residual** ("loss"): how far u is from satisfying the discretised PDE + BCs at each unknown. Target r = 0.	$r_j = 9(u_{j-1} - 2u_j + u_{j+1}) + 2$	curl of momentum (pressure eliminated), BDF2 in time
S = ∂r/∂u	Physics Jacobian: the same matrix a classic CFD solver builds.	[[−18, 9], [9, −18]]	sparse, from the stencils
J = SΦ	Jacobian of r with respect to z: how each residual reacts to each output weight.	[[6.854, 13.709], [−13.709, −6.854]]	2,162 × 4,323
$K = JJ^T + \lambda I$	Kernel matrix used when there are more weights than equations; factorised by Cholesky. Gives the smallest-norm Δz.	(J square: solve $J\Delta z = -r$ directly)	2,162 × 2,162, dense; the GPU-memory limit
Newton step	Linearise r around the current z and solve for Δz. Exact in one step for a linear PDE; repeated for Navier–Stokes.	1 step, r: 2 → 0	2–7 per time step, r drops ~2–3 decades each
ε, λ	Stopping tolerance on the relative update; small damping for numerical safety (Levenberg–Marquardt).	—	$\varepsilon = 10^{-8}$, $\lambda = 10^{-10}$ mean(diag K)

Why it reproduces CFD: the residual r is the same set of discrete equations a CFD code solves, and with at least as many hidden nodes as equations Φ can represent any field at the mesh points; so r = 0 gives the CFD solution, with the same mesh error. A different random seed changes z but not u. **Implementation note:** the code stores β with $A = G\beta$, $G = \Phi\Phi^T$ and $z = \Phi^T\beta$; then $K = SGS^T$ and Φ is never kept in memory. This is algebraically identical to lines 11–13.